TEMPORAL WORKFLOW DESIGN

Fulfillment Sync

Getting an approved sales order to the warehouse without waiting for a scheduled data copy, and knowing whether it arrived.

Describes the code in this repository. It runs on local simulators with synthetic data and has not been run against Oracle Fusion or Palantir Foundry.

1. The problem

After an order is approved in Oracle Fusion, warehouse staff can't see it in Fulfillment Manager for about 15–20 minutes. Oracle accepts the order immediately, then builds shipment lines in the background. Fulfillment Manager reads from Palantir Foundry, and those lines reach Foundry only through a scheduled CData snapshot of the whole table. The app receives no signal from Oracle at any point.

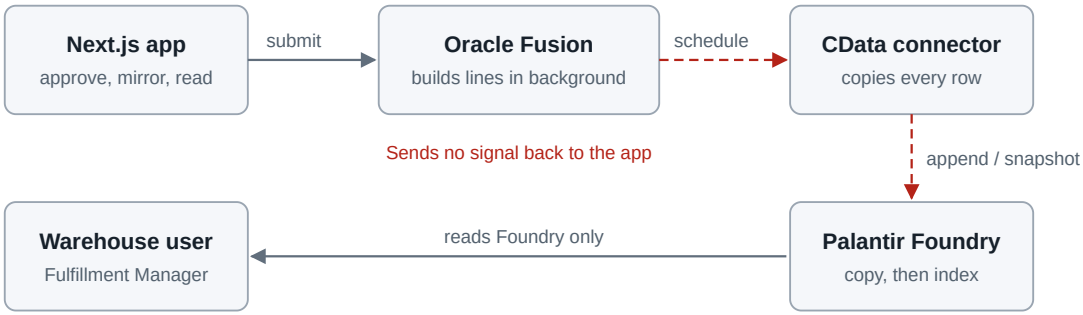


Figure 1. Today's path. Dashed lines run on a schedule.

Delay = Oracle processing + wait for next snapshot + snapshot run time + indexing

The client raised these issues on the architecture call:

Issue	What it means
Delay	Staff wait for the next snapshot, even when Oracle finished long ago.
Growth	Each snapshot reads everything, so it slows as data grows and uses the shared Oracle REST limit.
Silent failure	An order that doesn't reach fulfillment raises nothing. Finding out means a manual check in Oracle.
Constraints	Oracle populates too many fields to write shipment lines optimistically. UIs can't read Oracle live because of REST limits and paging. Analytics still need the snapshot.

This reproduces locally with no Temporal running. With the app and simulators in Docker, 5 seconds of Oracle processing and a 60-second snapshot, a browser test saw Oracle holding the line after 5.9 seconds, while the order reached Fulfillment Manager only after 64.1 seconds. On simulated time with a 15-minute schedule, the slowest order took 17.5 minutes with no existing data and 22.3 minutes with 100,000 lines. The schedules come from the client, but the other timings are assumptions, so these aren't Shift4 measurements.

2. The proposed solution

One durable Temporal workflow follows each approved order and **reconciles** it. Once Oracle's shipment lines cover the order, one operation reads both systems, writes only what differs, and repeats until they match. That single comparison handles retries, restarts, late changes and verification, and it also answers "why isn't this order here?" without opening Oracle. A scheduled sweep finds any approved order that was missed. The existing snapshot keeps running for analytics and correction, and could later run less often. Release for picking, create shipment, carriers and invoices aren't part of this phase.

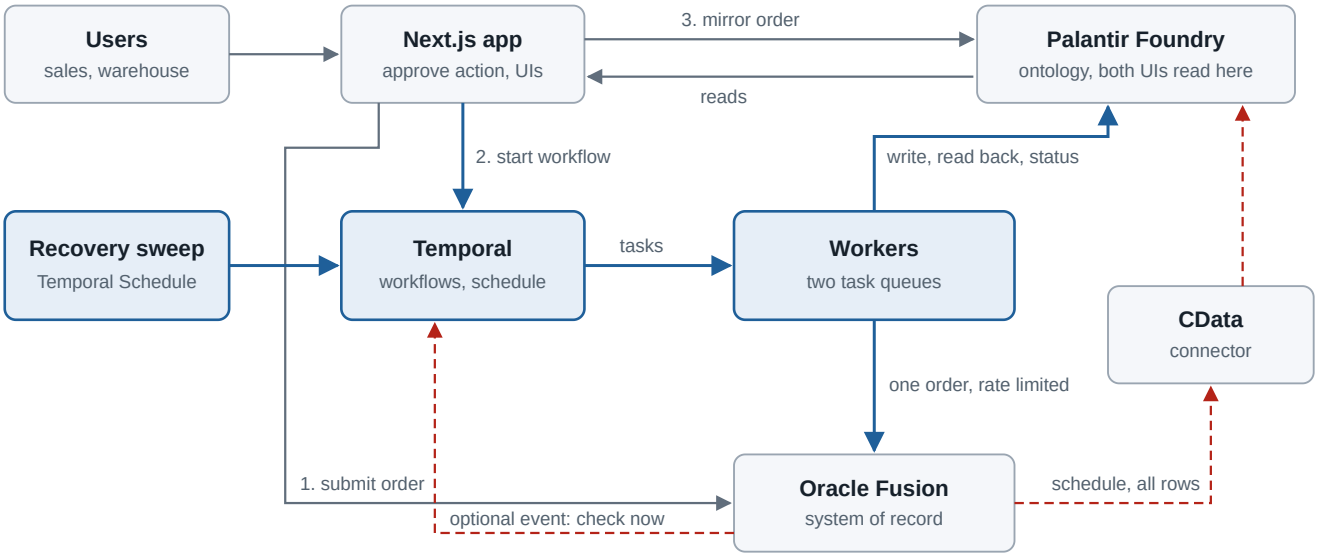


Figure 2. Blue marks what this design adds. Dashed lines are scheduled or optional. Workers reach Oracle and Foundry only through two adapters, which the client replaces with their own helpers. Oracle Fusion, Foundry and CData are simulated in this repository.

Issue	How the solution answers it
Delay	The order no longer waits for a schedule. It appears soon after Oracle creates its lines, so Oracle's own processing time is the floor.
Growth	Each read covers one order, so its cost doesn't depend on the size of the table. Oracle reads share a limit across workers.
Silent failure	Every order ends with a known status and reason, overdue orders raise an alert event, and one command explains any order from live data.
Lost responses, restarts, repeats	Workflow ID is known before any Oracle call, state survives worker restarts, and each pass reads fresh data and writes only differences, so repeating it is safe.
Customer data	Rows stay inside the activities, so Temporal history holds only keys, quantities and counts.
Rollback	`FULFILLMENT_SYNC` off returns the app to today's behaviour.

3. Workflow design

`deliverOrderToFulfillment`: one per approved order

The approve action submits to Oracle, starts this workflow, then mirrors to Foundry, so a mirror failure can't hold the order back. The workflow contains control flow only, and all I/O happens in activities. Its ID, `order-`

`delivery:<SourceTransactionNumber>`, is known before any Oracle call, so a lost response can't lose the order. `USE_EXISTING` makes a double click join the running workflow. `ALLOW_DUPLICATE` lets the sweep or a retry start again after a run has closed.

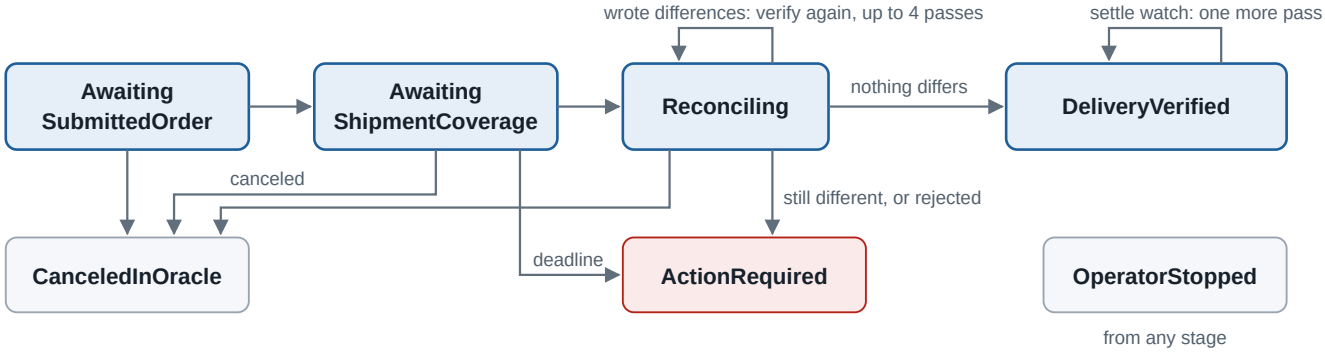


Figure 3. Stages, held in the `DeliveryStage` search attribute. `DeliveryOverdue` is a separate flag raised by a timer, so an overdue order still shows where it is, and work continues.

Activities

Activity	What it does	Uses	Queue and retry
`findSubmittedOrder`	Reads the order by source number. Fails until Oracle returns it as submitted, which covers a lost submit response. Returns its number and active lines.	Oracle: `findOrder`	`oracle-reads`. Polling, until the deadline
`checkShipmentCoverage`	One check per attempt. Fails until active shipment lines cover every ordered quantity, across all pages and warehouses. Every fifth attempt it re-reads the order to notice a cancellation.	Oracle: `listShipmentLines`, sometimes `findOrder`	`oracle-reads`. Polling, sharing the same deadline
`reconcileOrder`	The core. Reads the order, Oracle's lines and Foundry's rows, writes rows that are missing, older or changed, and removes rows Oracle no longer has. Returns only counts.	Oracle: both reads. Foundry: read, `sync-shipment-lines`, `remove-shipment-lines`	`oracle-reads`. 30-minute window
`recordDeliveryStatus`	Sends an audit or alert event, and saves the stage on the sales order. A failure here never changes the result.	Event sink. Foundry: `record-fulfillment-sync`	`order-delivery`. 3 attempts
`sweepMissedOrders`	Starts recent submitted orders with no final status. Skips running and stopped ones, and stops after 5 runs per order.	Recovery index. Temporal client	`order-delivery`. 3 attempts

`recoverySweepWorkflow`: the optional safety net

A Temporal Schedule runs it every 15 minutes, looking back 24 hours and covering at most 100 orders. It catches what a single request can't: Temporal unavailable at approval, or a process that crashed between submit and start. Overlap is set to Skip, so a slow sweep can't pile up. Without it, the snapshot remains the fallback.

Temporal features used

Feature	Use
Activity retries as polling	From 5 seconds, growing 1.5x to 60 seconds. No history growth, no held worker slot, and the UI shows attempts and the last result.
Timeouts	30 seconds per attempt, a 4-hour deadline for finding and waiting, and a 30-minute window that rides out a Foundry outage.
Failure types	A 429 honours `Retry-After`. A 401 or 403 keeps retrying, so an expired credential doesn't fail every open order. A rejected request or unexpected error stops at once.
Task queue rate limit	Holds Oracle-reading activities to one limit across all workers.
Durable timers	Raise the overdue flag, and run the settle watch that catches a change made soon after delivery.
Signal, Signal-with-Start	An optional Oracle event wakes a waiting order to check at once, or starts one that approval missed. It's only a hint, so a lost event only costs time.
Search attributes, query	`DeliveryStage`, `DeliveryOverdue` and `SourceOrderNumber` find orders by situation. `getDeliveryProgress` returns the current state.
Schedule, cancellation, replay test	The Schedule runs the sweep, an operator can stop a run cleanly, and a test replays recorded history to catch breaking code changes.
SDK metrics	Prometheus metrics, where rising schedule-to-start latency shows workers falling behind.

Failures and edge cases

What goes wrong	What happens
Oracle rejects the submit	No workflow starts, and the user sees the error. A 429 tells the user Oracle is busy.
Submit response lost, or a 5xx	Workflow starts as `uncertain` and trusts what Oracle holds.
Submit never reached Oracle	Keeps checking, then ActionRequired. Never a false success.
Temporal unavailable at approval	Approval succeeds, `NotStarted` is recorded, and the sweep starts it later.
Foundry mirror fails after submit	Workflow has already started, so the order is still delivered.
Worker restarts during a wait	Another worker continues from history, with no duplicates.
Foundry outage, or a lost write response	Retries, and the next read shows what was really written, so nothing is written twice.
Oracle changes between read and write	The next pass sees Foundry is older and writes the current rows.
Same timestamp, different data	Content is compared too, so the row is written.
Line added, or order canceled, during the run	Line added: delivery waits until it's covered. Canceled: ends as CanceledInOracle, including just before reconciling.
Row in Foundry that Oracle no longer has	Removed, and a later snapshot doesn't bring it back.
Expired credentials	Keeps retrying, and recovers once they're fixed.
A bad date or programming error	Stops at once with the reason, rather than looking like a long wait.
Lines never appear	Overdue at 30 minutes, then ActionRequired at 4 hours, with the reason recorded.

What goes wrong	What happens
An order keeps failing	The sweep stops restarting it after 5 runs, and never restarts a delivered or stopped order.

4. Pros and cons

Pros

Benefit	Why it matters
No wait for a schedule	The order appears soon after Oracle creates its lines. With the same containers and settings, approve to visible went from 64.1 seconds to 8.9 seconds, which is Oracle's 5 seconds plus the wait for the next check.
Timing no longer depends on luck	On the simulators, the wait varied from 5.5 to 20.5 seconds with the moment of approval today, and stayed at 2.1–2.2 seconds with the workflow.
Cost follows the order, not the table	Each read covers one order, so this path doesn't slow down as data grows.
One simple, repeatable core	Retries, restarts, late changes and verification all go through one comparison, and repeating it is always safe.
Every order has an answer	Each ends with a known status and reason, overdue orders raise an event, and `make explain` answers from live data with no manual check in Oracle.
Delivered means checked	The status is set only when a pass finds both systems match, and the settle watch checks once more.
Customer data stays out of Temporal	History holds keys, quantities and counts, so no Payload Codec is needed for this and large orders can't hit payload limits.
Survives failures	Worker restarts, lost responses, Foundry outages and expired credentials pause the work rather than lose it.
Ready for Oracle events	If Oracle's events cover this status, they cut the wait with no redesign, and polling remains the fallback.
Low-risk rollout	One flag returns the app to today's behaviour, the sweep is optional, and the snapshot remains the safety net.

Cons

Drawback	Effect	How to handle it
Not run on real systems	Adapters, Foundry actions and all timings are still assumptions.	Implement the adapters on the client's DEV environment
Oracle time is the floor, and unknown	The workflow can't read lines Oracle hasn't created, so no latency can be promised yet.	Measure each step on DEV
Two writers to the same data	Foundry compares the edit's time with Oracle's UTC timestamp, so a clock difference could hide a change from the snapshot. The settle watch catches this only soon after delivery. These rules come from Foundry's documentation and aren't tested on a real tenant.	Confirm timestamp type and clock difference, and test on DEV
Rows deleted after the workflow ends	Foundry keeps an object an Action created when its source row disappears, so it stays until something removes it.	Learn whether Oracle deletes or only cancels shipment lines

Drawback	Effect	How to handle it
More Oracle reads per order	Each pass makes at least two Oracle calls. The limit counts activities, not HTTP calls, the app or the connector. A stricter limit also adds delay: 1 activity per second is part of the 8.9 seconds above.	Get the real limit, then tune the rate and polling
New parts to run	Temporal and workers need operating, and the team needs to learn them.	Start on EC2 or Temporal Cloud at low volume
Integration work	The client implements two adapters with six methods, creates three Foundry actions, and adds a service identity. A per-row version check may need a function-backed action.	Use the simulators as a reference, and skip the sweep at first
Alerts need a destination	Overdue and ActionRequired events can post to a webhook, but no channel is chosen yet.	Choose one with the client
Oracle events aren't connected	The Signal path is built and tested, but nobody has confirmed which Oracle status means shipment lines exist.	Confirm with the client, or keep polling
Holds aren't recognised	An order on hold polls until the deadline, then shows ActionRequired.	Learn how holds appear in Oracle
Not production hardened	No workflow versioning yet, and the worker bundles its code at startup. History lasts 30–90 days unless exported through the event sink.	Add versioning and a prebuilt bundle before the first release